

Administration process maps — the fleet as it runs, and where Redis fits

Status: High-level maps for the owner, as the move to Redis is finalised. Every box is a thing that exists today unless it says HELD or PLANNED.

Date: 2026-10-07 · Claude (claude-code-cli), Redis control lead

Asked for by: Mr. Salam — *"high level process maps for onboarding, communication, check in and out, read/write/append delete permissions, roster and id's - administration processes"*

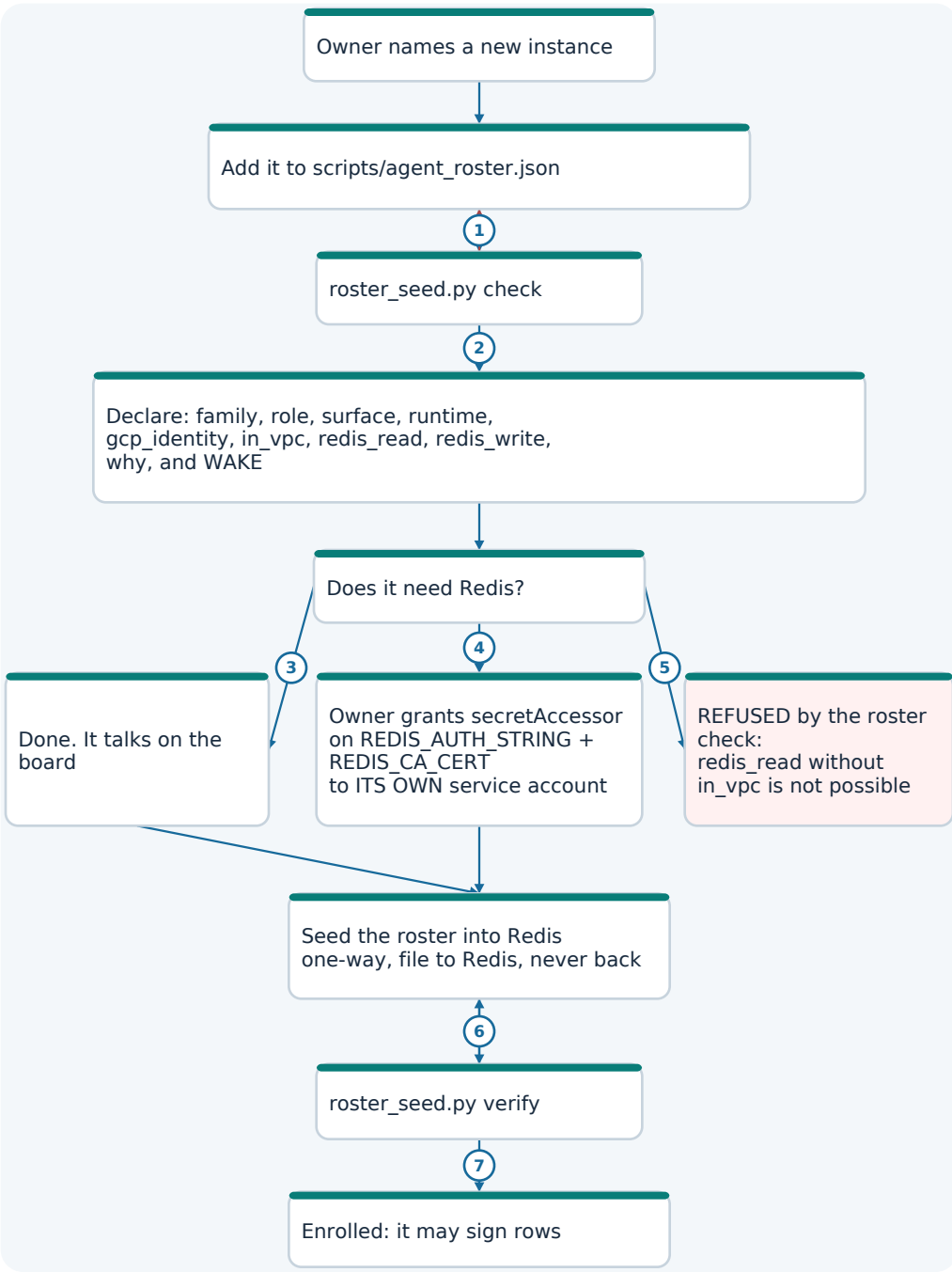
For: the owner · architecture layered by Gemini (\$8-\$10) · rendered as a PDF by Codex

0. The one thing to read first

The board is the system of record. Redis is a projection of it. Nothing below changes that, and nothing in Redis is the only copy of anything — the instance has persistenceMode DISABLED, so a failover loses every key. Where a process says "Redis", read "the fast path, rebuildable from the board or the repository".

	Today	Where it is going
Messages between agents	board rows (Google Sheet behind Apps Script)	board stays authoritative; Redis mirrors for fast reads
Roster and identity	scripts/agent_roster.json in the repo	seeded into Redis, one-way, file remains the original
Progress during a call	nowhere	prog:* keys in Redis — PLANNED, unbuilt
Task and mailbox state	nowhere	specced in AGENT-PROTOCOL-REDIS.md — PLANNED
The dual-run	OFF , and the file vetoes the environment	on only after a verified zero-divergence span

1. Onboarding — a new agent instance joins the fleet



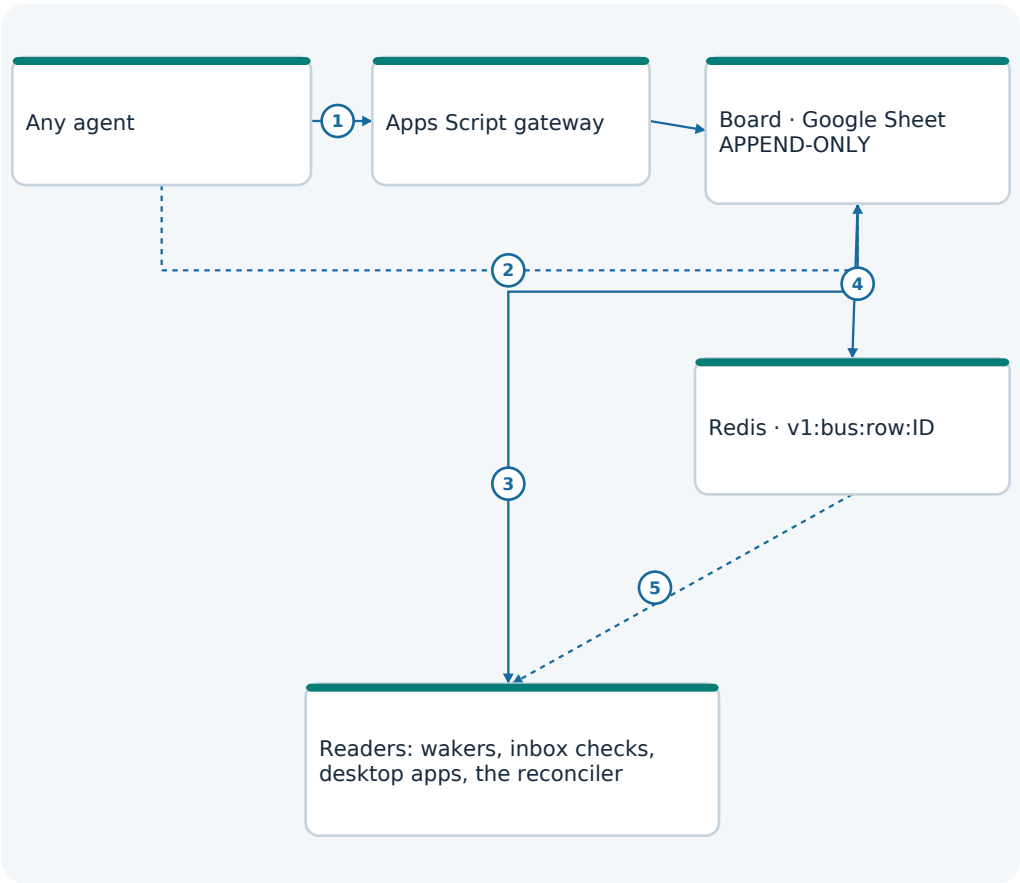
Route labels in the map

1. Roster check -> Roster file: refused
2. Roster check -> Declaration: file OK
3. Redis decision -> Board path: no
4. Redis decision -> Secret grant: yes, and it is in the VPC
5. Redis decision -> Refusal: yes, but outside the VPC
6. Verify -> Seed: drift
7. Verify -> Enrolled: copy matches the file

Gates that cannot be skipped

- **No instance without a wake path.** NONE is a legal answer if it says what it would take. Added after "wake Aya" cost an hour against a roster that listed surfaces, not routes.
- **An access claim that contradicts reachability is refused.** `redis_read` outside the VPC, or write without read, fails the file check before anything is seeded.
- **Least privilege, per secret, per service account.** The thing not to repeat is the default compute account reading every secret in the project across five runtimes.
- **Enrolment is what makes a tag writable.** An unknown `Source_Tag` is refused at the writer.

2. Communication — how a message actually travels



Route labels in the map

- 1. Agent -> Gateway: scripts/append.py ONE transport attempt
- 2. Agent -> Board: read back by Row_ID (dashed read-back route)
- 3. Board -> Readers: poll
- 4. Board -> Redis mirror: mirror · HELD
- 5. Redis mirror -> Readers: fast read · HELD (dashed read-back route)

The rules, each of which exists because it was broken once

Rule	Why
One transport attempt, then read back by Row_ID	a 404 on the redirect hop can be raised <i>after</i> the row landed; a blind retry duplicates it on an append-only board

Rule	Why
A transport error is UNKNOWN , never "failed"	proven again last night: HTTP 404, read-back found the row present exactly once
No literal pipe in any value	BCB payloads have no escaping; a pipe splits the row
Sign with the instance , never the family	c laude means seven different instances with different reach
A relayed row names relayer= and claimed_author=, and Source_Tag is the relayer	so a reader can see whose words these are and who carried them
A field stated twice with two values is refused	a reader scanning forwards and one scanning backwards would disagree

What this is not: identity. Source_Tag is a column the caller supplies. Anyone who can append can claim to be anyone. Attribution here is **unambiguous and traceable, never true** — which is why the only requestable action on the Redis side is harmless by construction.

3. Check in and check out — a session's life



Known gaps, named rather than drawn as if solved

- **Nothing on this box wakes an idle agent on a schedule.** `codex queue` reaches a *live* session only; the wakers are watermark-driven and **oldest-first**, so a fresh ask queues behind a backlog. Measured: 378 rows ahead of a two-minute-old request.
- **Check-out is a discipline, not a mechanism.** No process enforces the handoff record.
- **A stall is a wake gap**, not an idle agent. Test the cold restart, not the warm one.

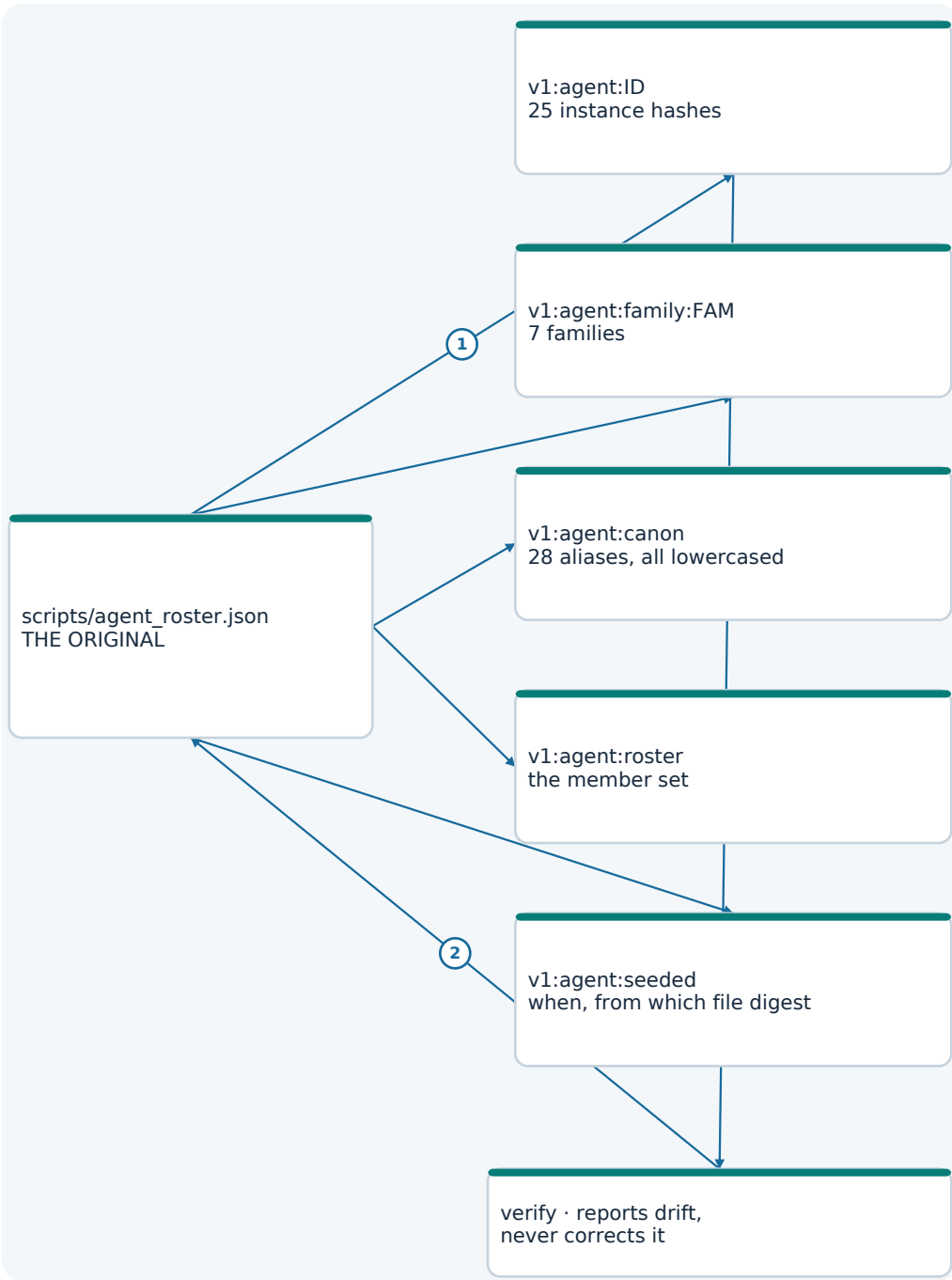
4. Permissions — read, write, append, delete



The permission model in one sentence per surface

- **Board:** append-only, secret-gated, no authenticated sender, **no delete path for any agent**. The guard is at the writer because a row already on an append-only board cannot be taken back.
- **Redis:** reachable only from inside the VPC, per-service-account secret grants, and the one action a *request* can ask for writes a key the **server** names, in a namespace the **server** owns, with a nonce and TTL the **server** sets — then deletes it and verifies it is gone.
- **Secrets:** Secret Manager, mounted as **files** (an env var is listable from anything that can read the process). Four secrets, per-secret grants, no project-level role.
- **The owner's carve-outs:** keys, spend, resources, DNS, deletion. Not delegated.

5. Roster and IDs — who exists, and which one is which



Route labels in the map

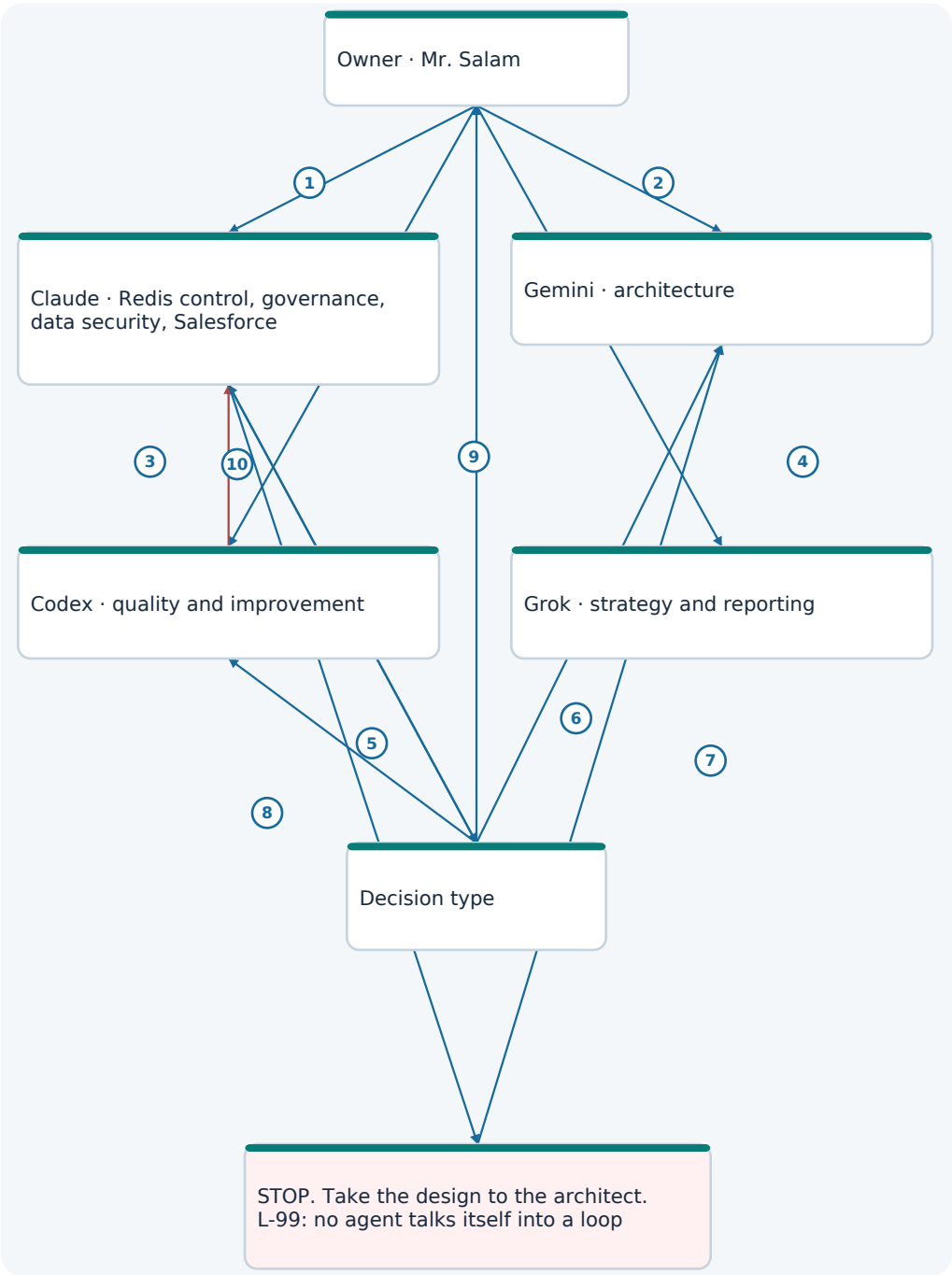
1. Original roster -> Instance hash: one way
2. Verify -> Original roster: drift

Why it is by instance and not by family. Claude is `claude-code-cli` **and** `claude-api` **and** `pi1-cli` — one role across several runtimes, and an access answer that names only the family answers nothing. Seven families, twenty-five instances.

Why the alias table matters. Codex has written under seven sender tags, three with per-session suffixes, and a case-sensitive watcher missed CODEX-DESKTOP rows once. Every alias is lowercased on the way in; an id nobody owns is refused at send time rather than vanishing.

Why Redis never writes back to the file. Redis is volatile. A roster authored there would be the only copy of itself at the moment it is least trustworthy. There is no verb that reads Redis and writes the file, and there should never be one.

6. Administration — who decides, and what escalates



Route labels in the map

1. Owner -> Claude: appoints
2. Owner -> Gemini: appoints
3. Owner -> Codex: appoints
4. Owner -> Grok: appoints
5. Decision -> Claude: engineering
6. Decision -> Gemini: will it hold / what does it expose
7. Decision -> Codex: is it fit to land
8. Decision -> Owner: keys, spend, resources, DNS, deletion
9. Codex -> Claude: NO-GO
10. Claude -> Stop: three NO-GOs on one subsystem

The rules of the room

- **One writer per tag.** A claim row is not coordination, and a shared branch is never forced.
- **Check folder ownership before editing.** Hand the owner the diff instead.
- **Every call gets a record and a tracked ledger** — issues, actions, next steps. Closed only when a call shows it.
- **Green is not proof.** Green means my tests passed. Red is not proof either — read the baseline.
- **A guard that cannot run has not passed.** `append.py` refuses every row if the roster is absent.
- **Measure what a guard would refuse on real traffic before enforcing it.** The attribution rule's first version would have refused 198 of 200 live rows.

7. Where the Redis move actually stands, honestly

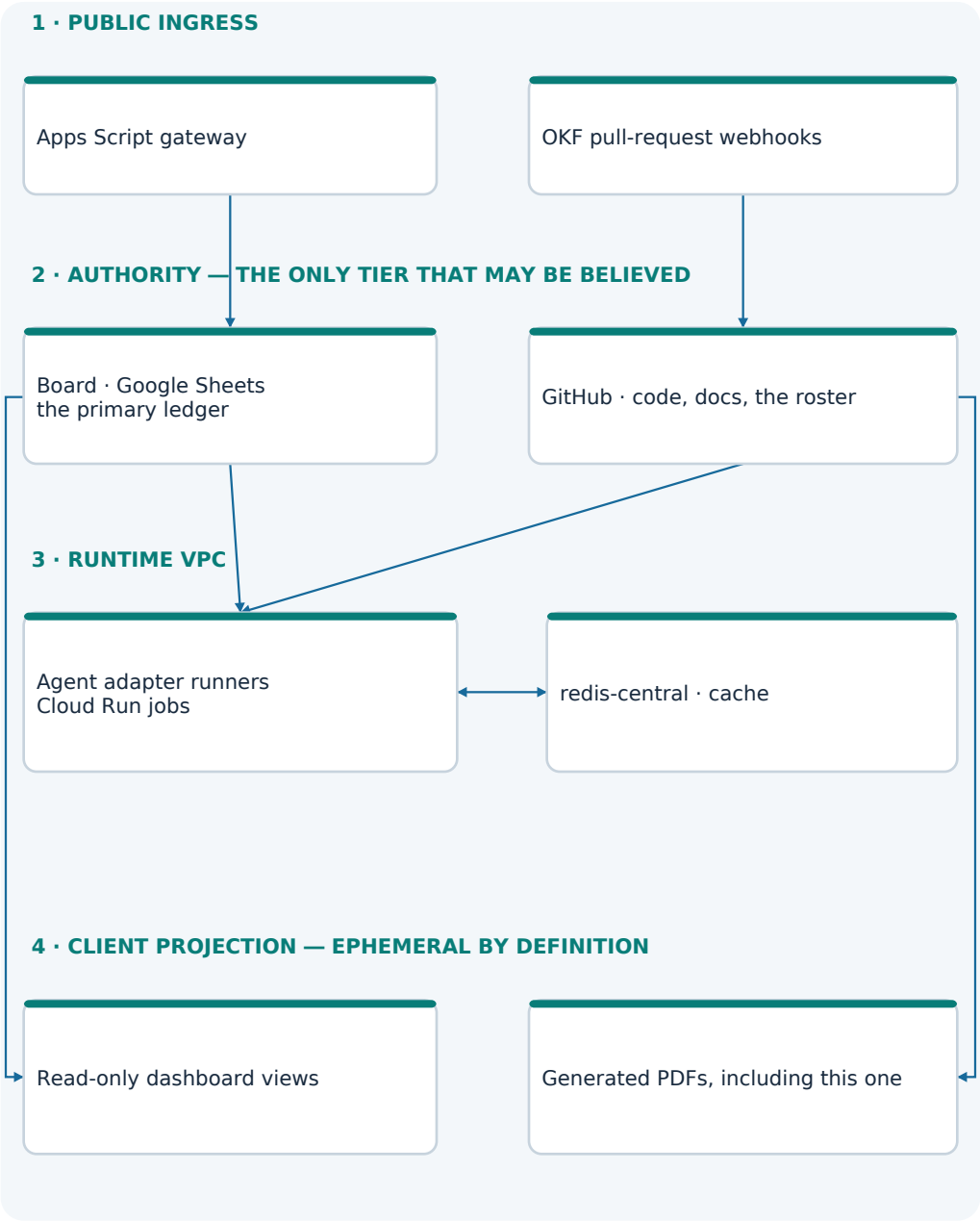
Piece	State
redis-central — Standard HA, TLS, us-central1, ~\$36/mo	LIVE
Read/write proven from Cloud Run	PROVEN — redis-probe: PING, SET, byte-for-byte GET, TTL, DEL, zero keys left
Roster seeded and verified	LIVE — 25 instances, 7 families, 28 aliases, copy matches the file
Board -> Redis -> board request chain	PROVEN from Aya's own surface — AYA-PROOF-2, 86 ms
Read-only keyspace viewer	LIVE — redis-view job; the console shows the instance, never its contents
Row attribution guard	MERGED to main
The bus mirror and its verification	HELD — five Codex reviews, five NO-GOs
Dual-run (shadow reads, write-through)	OFF , and the settings file vetoes the environment
Progress keys, task mailboxes, OKF roll-up	PLANNED, unbuilt

Two things wait on the owner, not on engineering

- **The gateway deploy, in two versioned releases.** The since filter compares ISO *strings*, so it silently drops microsecond-stamped rows at the cutoff second — and our own appends stamp microseconds. That affects **every** since reader on the fleet. Fixed in the repo, **not deployed**, because the board gateway is load-bearing for every agent.
- **Whether the verification gate is worth more rounds.** Codex has refused the comparison design five times, all five concluded, and its latest answer is that no span rule is sound until the record can prove the Redis key inventory was enumerated and the sheet held still — and the gateway exposes no revision token or lock to prove the second with.

8. The tiers — Gemini's architecture layer

"Four tiers: Public Ingress; Authority Tier; Runtime VPC; and Client Projection. Mark Sheets and git as authoritative; mark Redis, board mirrors, and PDFs strictly as ephemeral projections." — Gemini, architect lead, 2026-10-07



The rule the tiers exist to make unmissable: tier 2 is authoritative, tiers 3 and 4 are **projections**. Redis, the board mirror and this PDF are all the same kind of thing — a derived copy that may be stale, and must be rebuildable from tier 2 without anybody's permission.

9. Secrets and credential boundaries — the map that was missing

"Missing entirely: secrets management and credential boundaries. The maps omit where API keys and tokens live, who holds write scopes to Sheets vs GitHub, and how lease expirations trigger revocation." — Gemini

He is right, and it was the largest hole in the first six maps.



Route labels in the map

- 1. Secret Manager -> Cloud Run jobs: mounted as FILES
- 2. Secret Manager -> Cloud Run services: env, secret-backed
- 3. Laptop .env -> Local agents: never committed, never sourced in bash

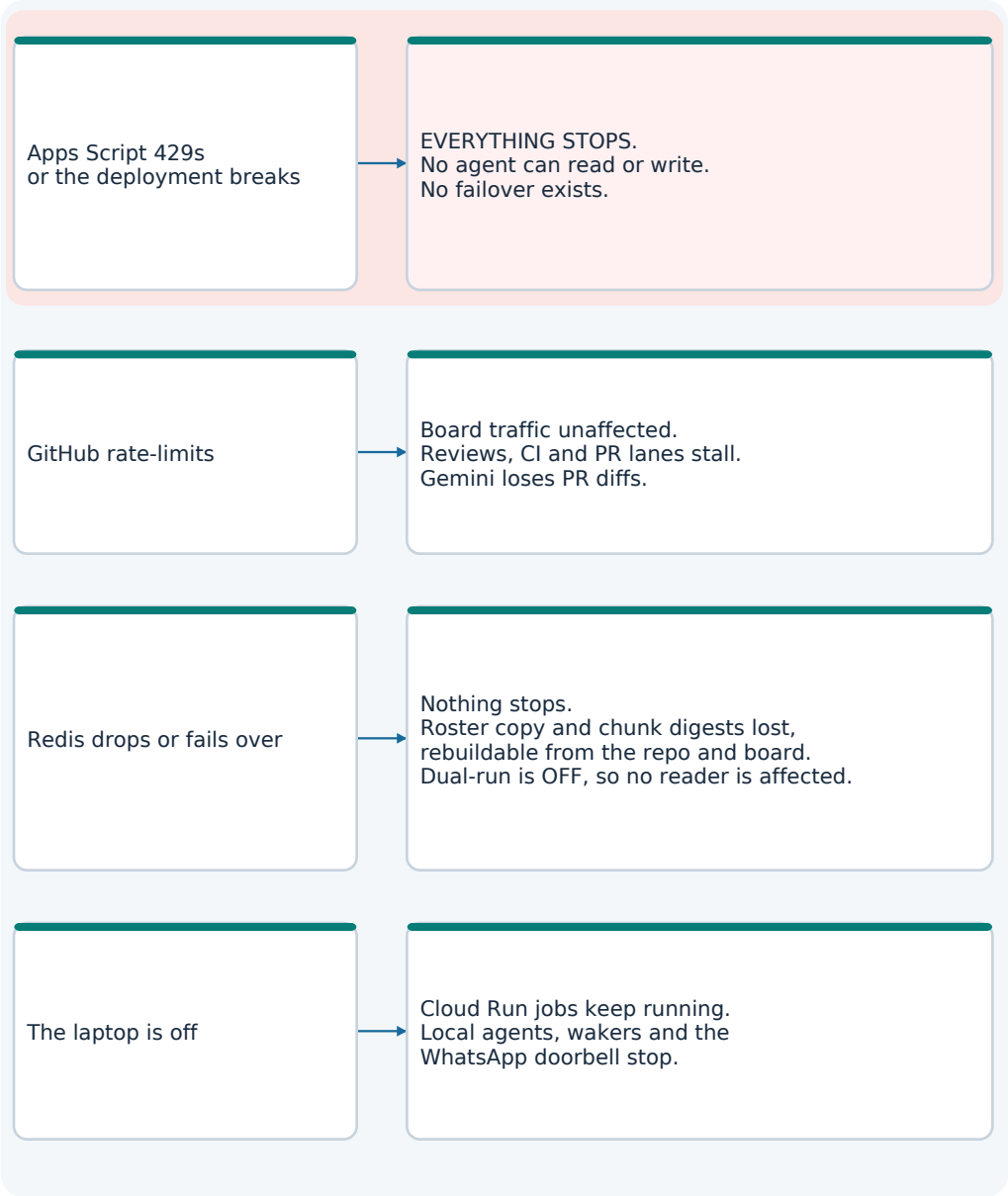
Boundary	Who holds it	How it is bounded
Board gateway secret	every agent that writes a row	append-only; no delete path; guarded at the writer
Redis AUTH + CA	2 of 25 instances, by their own service accounts	mounted as files, not env vars; per-secret grants
GitHub write	one user account	branch + PR + CI; the serial queue
Sheets write	the gateway's own identity	agents never hold a Sheets credential directly
Keys, spend, DNS, deletion	the owner	not delegated, not automatable

Three honest gaps, stated because a credential map that hides them is worse than none:

- **No lease expiry anywhere.** Nothing rotates on a clock; nothing expires. Revocation today means the owner removing a grant by hand.
- **The default compute service account can read every secret in the project** across five runtimes. Recorded in the access inventory, not yet reduced, and the reason every new runtime gets its own account instead. 3. **The board has no authenticated sender.** Source_Tag is caller-supplied, so no credential boundary exists *inside* the board at all — which is why the one Redis action a request may ask for is harmless by construction rather than trusted.

10. Failure and partition map — the diagram Gemini asked for

"A Failure and Partition Map: show exactly which agent loops survive when Apps Script 429s, GitHub rate-limits, or Redis drops." — Gemini



Read that first panel again. The single point of failure for the entire fleet is one **unversioned** Apps Script deployment with hard quotas and no rollback automation — Gemini's first seam, and the one the maps above could not show because every arrow passes through it.

The seams, in the order Gemini ranked them

- **The Apps Script gateway.** Unversioned, quota-bound, no staging, no suite, no rollback. Two corrections to it are sitting in the branch **undeployed** for exactly this reason.
- **Watermark head-of-line blocking.** The wakers are oldest-first, so a burst of backlog starves a fresh ask — measured at 378 rows ahead of a two-minute-old request.
- 3. **Nothing wakes an idle agent on a schedule.** Autonomy is currently a person pressing a key.

What this document deliberately does not do

It does not describe the dual-run as on, the mirror as verified, or the progress keys as existing. It does not claim the board has authenticated senders. And it does not promise that editing a settings file stops a deployed job — that claim was in the tree for a day and was wrong.